

60.3. La fonction `teindrePanneau()` — une case du panneau dans une palette

`#include <teindrePanneau>` : `teindrePanneau(colonne, ligne, palette)` teint une case du panneau (Game Boy Color).

1. Ce qu'il faut comprendre

Ce qu'elle fait : comme `teindre()`, mais **dans le panneau** : un cœur rouge dans le HUD, un mot important dans un dialogue.

La ligne à écrire : `#include <teindrePanneau>`.

Ses arguments : la colonne et la ligne du panneau, la palette.

Ce qu'elle coûte : environ **100 octets** la première fois.

Essaie : teins les cinq lettres de VIES avec une boucle `for`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **325 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	62 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte	– attend de pouvoir écrire une case
EcrireTexte	10 o	← textePanneau()	– écrit un texte, lettre par lettre
CopierTuiles	58 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	64 o	← CopierTuiles	– 4 lettres sur 43
"VIES"	4 o	← textePanneau()	– un texte gravé

Lettres gravées : EISV

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

teindrePanneau()	× 1	17 o sur place, dans main()
panneau()	× 1	15 o sur place, dans main()
couleurFond()	× 1	12 o sur place, dans main()
textePanneau()	× 1	11 o sur place, dans main() + EcrireTexte (10 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

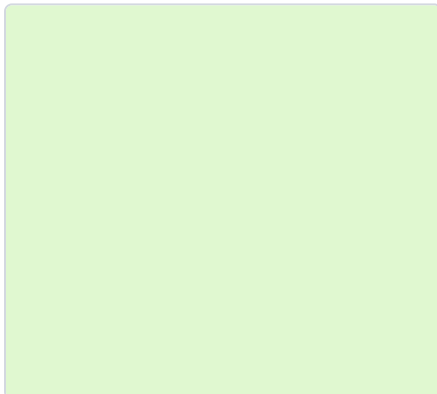
```
// ---- #include <teindrePanneau> : une case du panneau dans une palette ----  
  
#include <couleurFond>  
#include <textePanneau>  
#include <panneau>  
#include <teindrePanneau> // teindrePanneau() : la palette d'une case du panneau
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    couleurFond(1, 3, 31, 0, 0); // palette 1 : des lettres rouges  
    textePanneau(1, 0, "VIES");  
    panneau(0, 128);  
    teindrePanneau(1, 0, 1);      // le V, en rouge  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

VIES dans la bande du bas, avec le V en rouge.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la case (1, 0) du panneau est dans la palette 1 (1)
- ✓ la case d'à côté reste dans la palette 0

